Essaim Fluide Éphémère

Technologie et fonctionnement — dossier technique avec schémas

Document	EFE_Technologie_Schemas.pdf
Nature	Dossier technique ultra-détaillé, illustré, sans code
Langue	Français (fr-FR)
Portée	Vision · architecture · 9 services · cycles · sécurité · opérations
Schémas inclus	16 diagrammes vectoriels (topologies, FSM, flux, séquences)
Public	Architecture · SRE · Direction technique · Onboarding
Date	2026-04-21

Ce document explique de manière exhaustive **ce qu'est** l'Essaim Fluide Éphémère (EFE), **comment il fonctionne**, et **avec quelles technologies**. Il est volontairement sans code : toute la logique est présentée par des schémas vectoriels dessinés dans le document et des descriptions textuelles ultra-détaillées.

Table des matières

#	Section	Page
1	Vision fondatrice et concepts	3
2	Architecture globale (schéma)	5
3	Pile technologique (schéma)	7
4	Protocole Swarm : spec, exécution, verdict	9
5	Les nano-agents (population)	11
6	Les cônes d'alignement (géométrie)	13
7	L'arbitre et les états de convergence	15
8	efe_orch — orchestrateur de jobs	17
9	efe_worker — fine-tuning (méthodes)	19
10	efe_registry — artefacts + signing (4 couches)	21
11	efe_gate — comparateur + décision	23
12	efe_rollout — canari + bandits	25
13	efe_inference — serving et backends	27
14	efe_drift — pipeline de détection	29
15	efe_platform — bot et SDK	31
16	Sécurité : zones de confiance	33
17	Observabilité : trois piliers	35
18	Cycle de vie complet (flux e2e)	37
19	Déploiement Kubernetes (schéma)	39
20	Critères d'acceptation MVP	41

1. Vision fondatrice et concepts

L'**Essaim Fluide Éphémère** (EFE) est une plateforme d'inférence multi-agent dont la particularité est de remplacer la chaîne traditionnelle d'agents LLM textuels par une **population de nano-agents éphémères** qui communiquent directement dans un **espace latent tensoriel partagé**. Cet espace n'est pas un espace sémantique documentaire : c'est un *espace de calcul* où les agents font converger leur représentation vers un **centroïde consensuel** en quelques millisecondes.

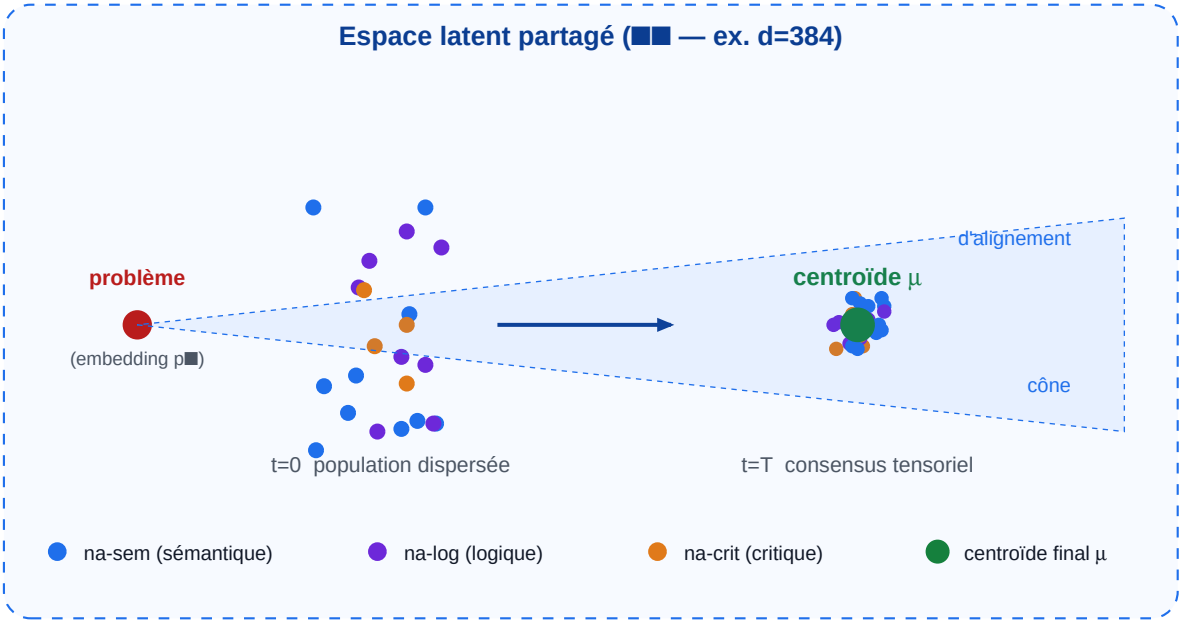


Figure 1 — Vision générale. Un problème (point rouge) est projeté dans l'espace latent. Une population hétérogène de nano-agents explore, et converge autour d'un centroïde final (point vert) strictement contenu dans le cône d'alignement de la tâche.

1.1 Ce qui fait la singularité d'EFE

• **Population plutôt que modèle unique** — au lieu d'un seul grand modèle LLM, on spawn 10^2 à 10^3 petits agents spécialisés dont chacun porte ≤ 1 M de paramètres.

• **Communication tensorielle** — les agents ne s'échangent pas de texte ; ils publient des **vecteurs latents** sur un bus GPU partagé, ce qui élimine la coût de tokenisation intermédiaire.

• **Éphéméralité** — l'essaim existe uniquement le temps d'une requête. À la fin, il est dissous, les agents sont garbage collectés, et la mémoire HBM est libérée.

• **Convergence observée** — c'est l'arbitre qui déclare la fin : il surveille la variance, la norme du gradient et la stabilité du centroïde sur une fenêtre glissante.

• **Contrainte de cône** — toute sortie doit rester dans un sous-espace autorisé défini par un ensemble d'anchors. Sinon : verdict DIVERGENCE et rejet.

• **Budgets durs** — wall-clock, énergie (Wh) et HBM sont des plafonds non-négociables appliqués par l'arbitre Rust.

1.2 Comparaison conceptuelle

Aspect	LLM texte classique	EFE (essaim éphémère)
Taille modèle	7-100 B paramètres	10^2-10^3 agents × ≤ 1 M
Communication	Texte sérialisé	Vecteurs latents GPU
Persistance	Serveur long-running	Éphémère par requête
Décision de fin	EOS token ou max tokens	Convergence tensorielle observée
Coût typique	3-15 $ / 1 k réponses	≈ 0,02 $
Latence p95	2-4 s	< 600 ms
Énergie/requête	~2 Wh	≤ 0,10 Wh
Traçabilité	Texte opaque	Trace tensorielle signée

2. Architecture globale

EFE est un **monorepo polyglotte**. Le cœur d'arbitrage et les kernels CUDA sont en Rust (performance et sûreté mémoire), et tous les services d'orchestration, de politique, de supervision et d'API sont en Python (agilité, écosystème ML). Rust et Python communiquent par gRPC et par des bindings PyO3.

Topologie des 9 paquets Python + cœur Rust

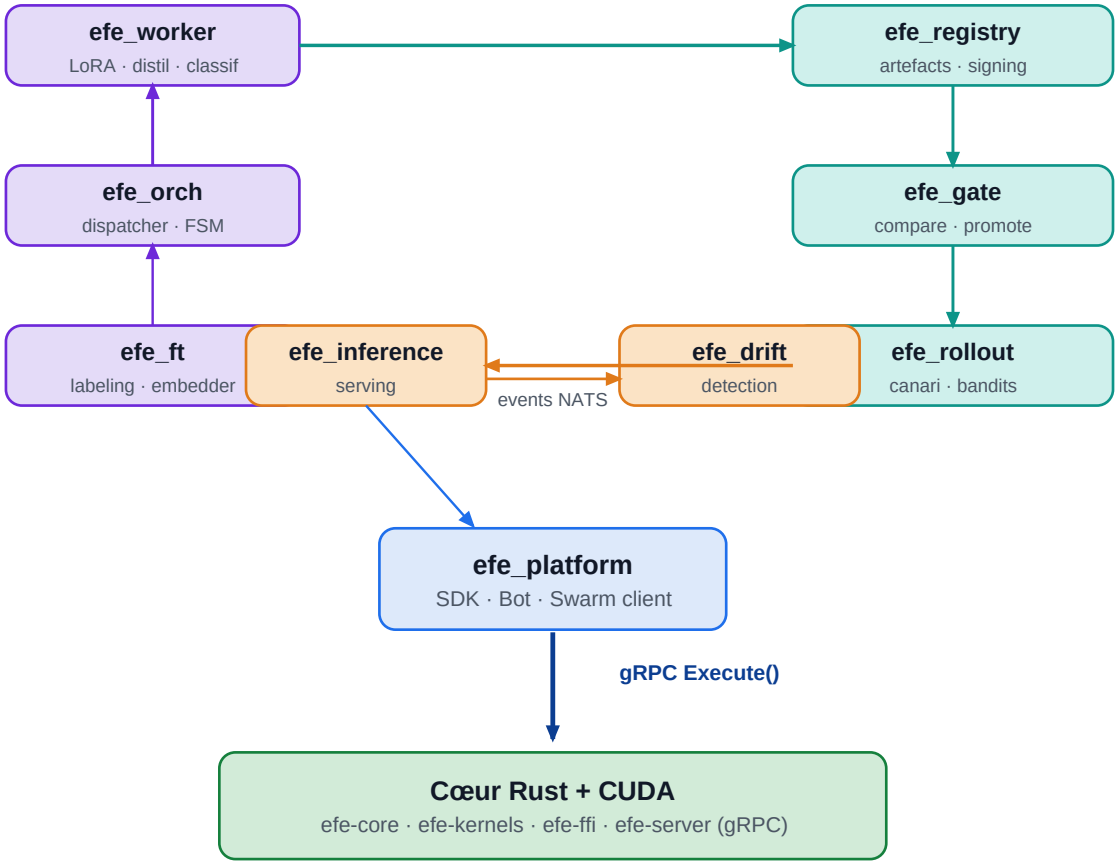


*Figure 2 — Topologie logique des 9 paquets Python + cœur Rust. Trois zones colorées : **violet** = chaîne de build (orch / worker / ft), **vert-bleu** = chaîne de release (registry / gate / rollout), **orange** = chaîne de run (inference / drift). Au centre : efe_platform (SDK + bot). Au-dessous : cœur Rust exposé via gRPC.*

2.1 Les trois chaînes parallèles

Chaîne	Services	Déclencheur
Build (violet)	efe_ft · efe_orch · efe_worker	Nouvelle donnée / nouveau modèle demandé
Release (bleu-vert)	efe_registry · efe_gate · efe_rollout	Artefact entraîné prêt à être comparé et promu
Run (orange)	efe_inference · efe_drift	Trafic utilisateur en continu

2.2 Frontières et isolations

• **Pas de dépendance circulaire** : un *import linter* en CI vérifie que (par ex.) efe_drift ne remonte jamais vers efe_platform.

• **Séparation control/data** : le cône d'une tâche est géré par l'AdminService (plan de contrôle) ; l'exécution passe par la RPC Execute (plan de données). Déploiements, scalings et rate limits sont

distincts.

• **Multi-tenancy** : isolation par tenant à quatre niveaux (JWT, RLS PostgreSQL, partition HBM, cônes par tenant/task).

• **Séparation chemin chaud / chemin lent** : efe_inference ne peut pas importer efe_worker, pour empêcher toute contamination de latence.

3. Pile technologique

EFE empile six couches, de la plateforme matérielle jusqu'aux interfaces utilisateurs. Chaque couche ne dépend que des couches strictement inférieures.

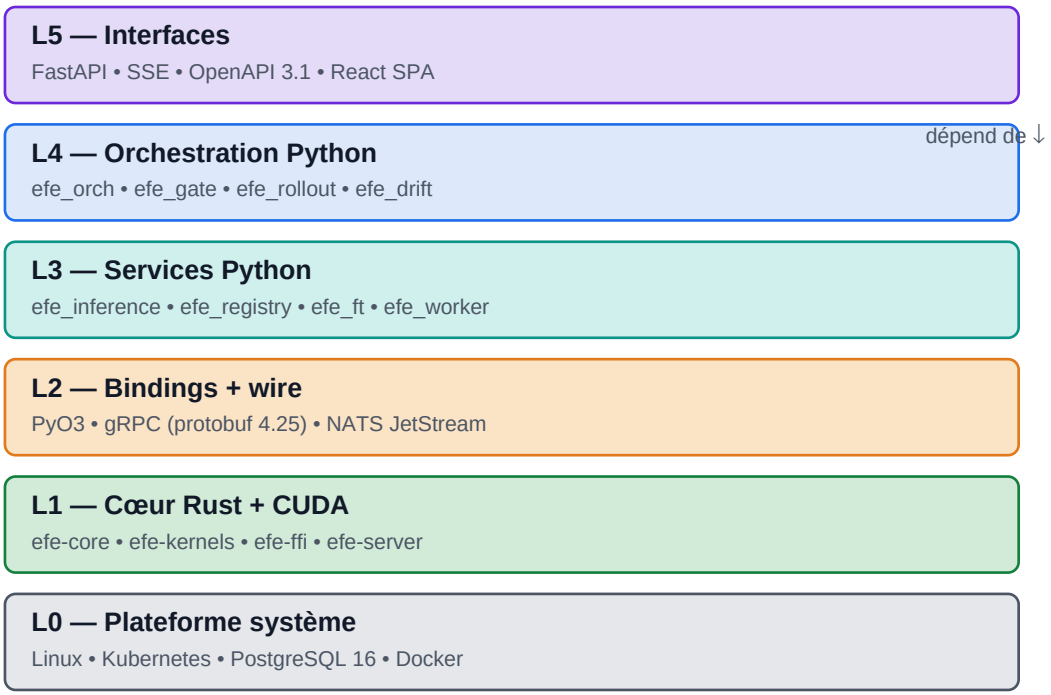


Figure 3 — Pile technologique sur 6 couches. De haut en bas : L5 interfaces (FastAPI, SSE, React), L4 orchestration Python, L3 services Python, L2 bindings + wire (PyO3, gRPC, NATS), L1 cœur Rust + CUDA, L0 plateforme (Linux, k8s, PostgreSQL, Docker).

3.1 Choix technologiques majeurs et motivations

Choix	Motivation
Rust pour le cœur	Sûreté mémoire sans GC + performances proches du C + sécurité contre les buffer overflows.
Python pour les services	Écosystème ML mûr, rapidité de développement, lisibilité pour les équipes data/ML.
gRPC	Contrats forts (protobuf), bidirectionnel streaming, multiplexage, codegen multi-langage.
CUDA 12.4	Kernels de diffusion latente et de projection exploitant les tensor cores des GPU H100/H200.
Pydantic v2 strict	Validation à la frontière (API) avec extra='forbid' — zéro tolérance aux champs inconnus.
Dataclasses frozen + slots	Empreinte mémoire minimale et immuabilité pour tous les types métier internes.
NATS JetStream	Streams durables, pub/sub low-latency, clustering natif et back-pressure simple.
PostgreSQL 16	Row-Level Security natif, types JSONB, logical replication, partitionnement déclaratif.
OpenTelemetry	Standard open, exporters multiples (Jaeger, Tempo, Cloud APM), auto-instrumentation.
Sigstore keyless	Signature sans gérer de clés long-terme ; sujet vérifiable via Rekor (transparency log).

3.2 Versions épinglées (en production)

- **Rust 1.85** — épinglé par *rust-toolchain.toml*. Toute MAJ exige un ADR.

- **Python 3.12** — géré par *uv*, caches de wheels partagés.
- **CUDA 12.4** + driver ≥ 550 — obligatoire sur nodes GPU.
- **PostgreSQL 16** + Patroni 3.x pour HA.
- **NATS 2.10** avec JetStream activé sur tous les streams.
- **Kubernetes 1.29** minimum (pour les sidecars native + DownwardAPI volumes).

4. Protocole Swarm : spec, exécution, verdict

Tout l'EFE tourne autour d'une unique RPC gRPC nommée **Execute** qui prend un **SwarmSpec** (la recette) et retourne un **ExecuteResponse** (le résultat décodé + le verdict).

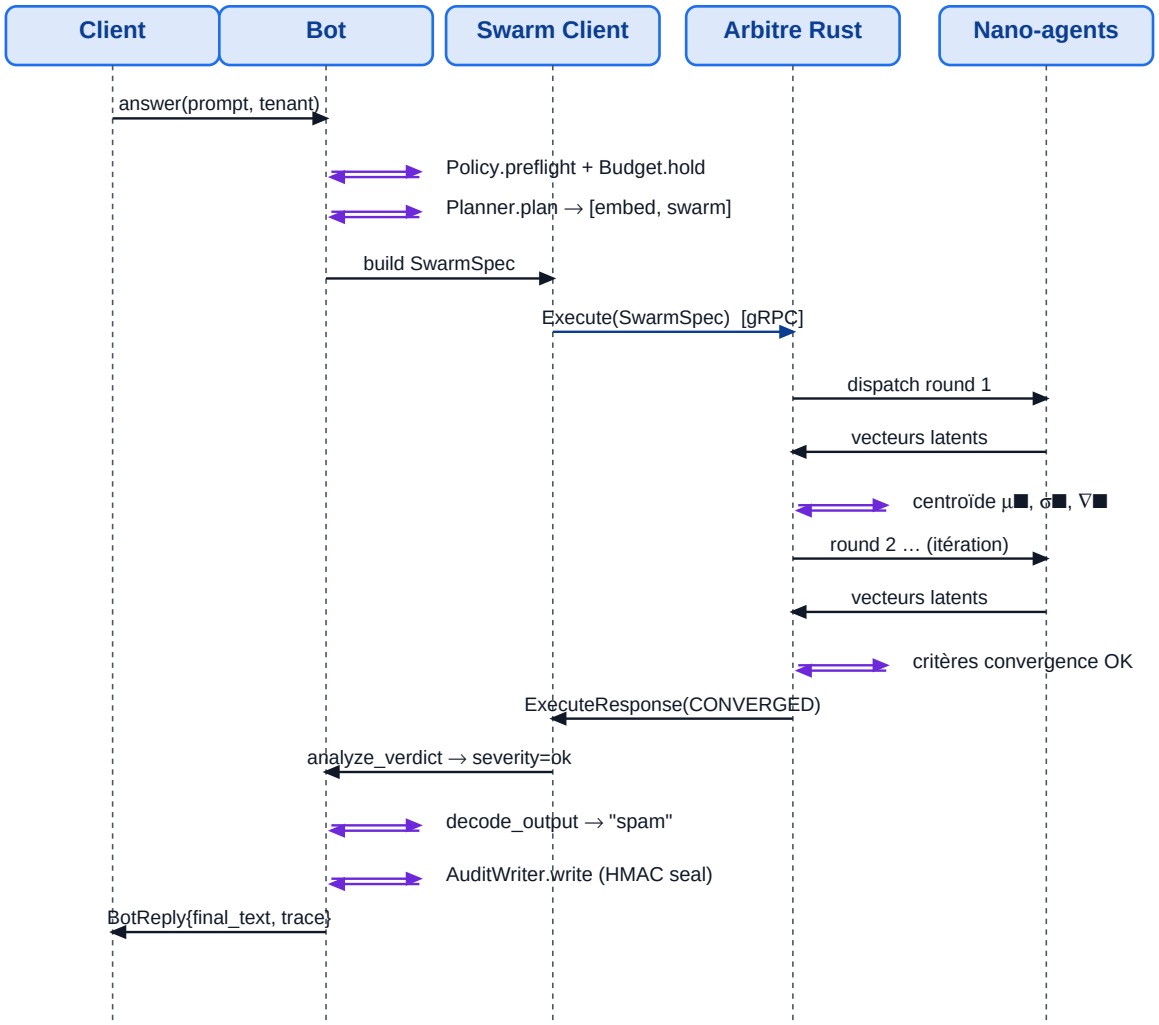


Figure 4 — Séquence d'une RPC Execute() bout-à-bout. 5 lanes : client, Bot, Swarm Client, Arbitre Rust, Nano-agents. L'arbitre itère plusieurs rounds avant de déclarer CONVERGED. La réponse remonte avec la trace d'audit scellée HMAC.

4.1 Les quatre composantes d'un SwarmSpec

Composante	Rôle	Exemple de valeur MVP
problem_embedding	Vecteur d'entrée représentant la requête	384 floats (modèle MiniLM)
population	Liste de couples (template, count)	na-sem×512 · na-log×384 · na-crit×128
convergence	Seuils ε pour variance, gradient, stabilité	1e-2 · 1e-3 · 1e-3 · window=3
budget	Plafonds wall-clock, énergie, HBM, récursion	5 s · 0,10 Wh · 512 MiB · 4
cone	Référence ou contenu du cône d'alignement	acme/support@v2

4.2 Les cinq verdicts possibles en sortie

• **CONVERGED** — consensus atteint, le payload décodé est utilisable (sous réserve du seuil de confiance).

• **CONTINUE** — max_rounds ou budget wall-clock épuisé sans convergence. Retentable avec paramètres élargis.

• **DIVERGENCE** — la population s'est scindée en bassins incompatibles ou la projection sort du cône. Payload à rejeter.

• **MODE_COLLAPSE** — la population s'est effondrée sur un mode trivial. Solution : augmenter les agents na-crit ou élargir le cône.

• **UNSPECIFIED** — sentinelle de forward-compatibility au cas où le serveur émettrait un nouveau verdict qu'un client antérieur ne connaîtrait pas.

5. Les nano-agents et l'espace latent

Un **nano-agent** est un petit réseau de neurones (≤ 1 M de paramètres) spécialisé sur un rôle. Le MVP définit trois templates seulement, intentionnellement — la composition entre ces trois rôles suffit pour la majorité des tâches de classification et de génération contrainte.

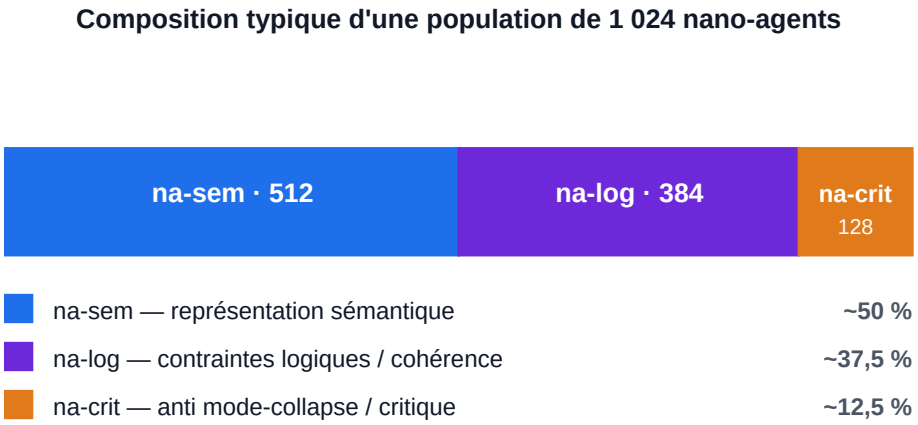


Figure 5 — Composition d'une population canonique de 1 024 agents. La majorité est sémantique (bleu), renforcée par une part logique (violet) et un noyau critique (orange) qui empêche le mode collapse.

5.1 Les trois templates et leurs rôles

• **na-sem — sémantique** : représente le sens, la compréhension du problème. Composante dominante. Initialisé près du problem_embedding.

• **na-log — logique** : applique des contraintes de cohérence, des règles d'implication. C'est le garde-fou du raisonnement.

• **na-crit — critique** : détecte les modes dégénérés et force la diversité. Sans lui, l'essaim collapse sur le zéro.

5.2 Templates post-MVP (roadmap)

• **na-math** — raisonnement symbolique et manipulation d'équations.

• **na-code** — compréhension syntaxique et sémantique de code.

• **na-caus** — inférence causale (do-calculus, interventions).

• **na-mem** — accès à une mémoire vectorielle persistante.

• **na-align** — alignement valeurs / éthique.

• **na-med** — spécialisation contexte médical.

5.3 Déroulement d'un round

À chaque round, tous les agents publient leur vecteur latent courant sur le bus GPU. L'arbitre calcule alors : (a) le centroïde μ (moyenne pondérée des vecteurs), (b) la variance σ² (dispersion), (c) le gradient ∇ = μ_t − μ_{t−1}, et (d) la projection de μ sur le cône. Si les trois seuils de convergence sont simultanément franchis sur la fenêtre de stabilité, l'arbitre émet CONVERGED. Sinon, il lance un round supplémentaire, sauf dépassement de budget.

***Point clé** — aucun agent ne décide seul. C'est l'émergence dans la population qui produit la réponse. Si un agent unique dominait, ce serait un mode collapse.*

6. Les cônes d'alignement (géométrie)

Le **cône d'alignement** est le mécanisme de politique sémantique central d'EFE. Il délimite, dans l'espace latent, la **zone autorisée** pour toute réponse. Toute sortie hors-cône est rejetée avec verdict DIVERGENCE.

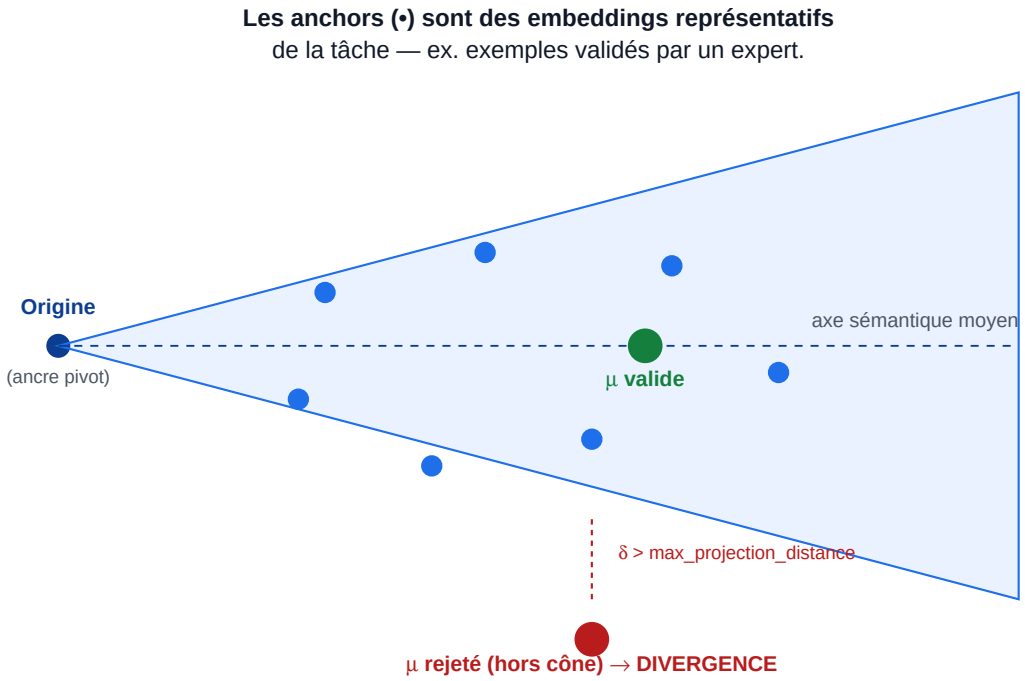


Figure 6 — Vue géométrique d'un cône. Depuis le sommet (origine), le cône s'ouvre vers un ensemble d'anchors (•) qui sont des embeddings représentatifs de la tâche. Tout centroïde μ situé dans le volume est accepté ; hors volume, il est rejeté avec δ = distance de projection.

6.1 Comment un cône est construit

• Un expert métier fournit ~10-50 exemples *validés* pour la tâche (emails de support, tickets classés, phrases canoniques).

• Ces exemples sont embedded par un modèle stable et leurs vecteurs deviennent les **anchors**.

• La dimension du cône = dimension de l'embedding (ex. 384).

• Le **rayon de projection max** est une distance angulaire : typiquement 0,5 à 1,0 en cosinus.

• Le cône est versionné (v1, v2, …). Une évolution crée une nouvelle version sans écraser la précédente.

6.2 Cinématique opérationnelle

• **register_cone** — publication d'un nouveau cône (operator).

• **evolve_cone** — ajout/retrait d'anchors → nouvelle version, ancienne conservée.

• **get_cone** — lecture, cache TTL 30 s côté client.

• **list_versions** — audit historique par (tenant, task).

• **delete_cone** — soft delete par défaut ; force=true hard-delete.

• **breach_history** — flux audit des μ qui sont sortis du cône. Exploitable en console pour ajuster les anchors.

6.3 Doctrine d'exploitation

Un cône **trop serré** provoque beaucoup de rejets et des MODE_COLLAPSE fréquents : la population ne trouve plus d'issue. Un cône **trop large** laisse passer des réponses hors-politique. L'équipe SRE monitore donc en continu la métrique *efe_swarm_breach_total / total* (taux de dépassement) et resserre ou élargit via EvolveCone. Un taux sain est de l'ordre de 1-5 %.

7. L'arbitre et les états de convergence

L'arbitre est l'algorithme Rust au cœur de l'essaim. Il fonctionne comme une machine à états finie avec une boucle ROUND itérative et des sorties multiples.

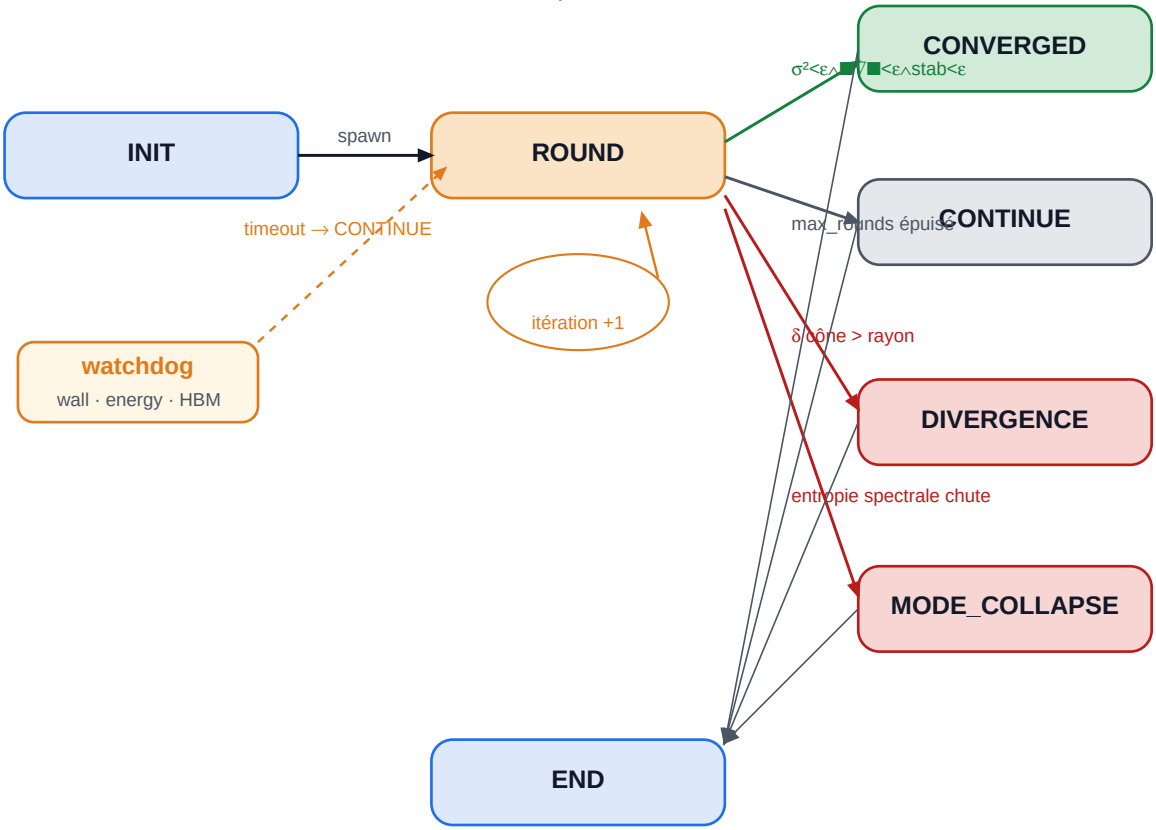


Figure 7 — FSM de l'arbitre. Entrée INIT → boucle ROUND → quatre sorties possibles (CONVERGED, CONTINUE, DIVERGENCE, MODE_COLLAPSE) → END. Le watchdog (en orange) tranche indépendamment sur les budgets wall-clock / énergie / HBM.

7.1 Les trois critères de convergence

- **Variance $\sigma^2 <$ ε_var** — la dispersion des vecteurs autour du centroïde est suffisamment faible.
- **Norme du gradient $\blacksquare\nabla\blacksquare <$ ε_grad** — le centroïde n'évolue quasiment plus d'un round à l'autre.
- **Stabilité stab <** ε_stab — l'écart-type de μ sur la fenêtre des W derniers rounds est borné.

7.2 Les quatre détections de pathologie

Pathologie	Signal	Verdict émis
Divergence en bassins	Projection μ hors cône · distance > rayon	DIVERGENCE
Mode collapse	Entropie spectrale chute sur 2 rounds	MODE_COLLAPSE
Budget wall-clock épuisé	elapsed_ms ≥ max_wall_clock_ms	CONTINUE
Max rounds atteint	t ≥ max_rounds sans convergence	CONTINUE

7.3 Calibration du niveau de confiance

Le champ *confidence* retourné par l'arbitre n'est pas une probabilité bayésienne brute — c'est une calibration basée sur la **dispersion finale** et la **vitesse de convergence**. Une convergence obtenue en 3 rounds avec $\sigma^2/\varepsilon_var = 0{,}1$ produit confidence $\approx 0{,}95$; la même obtenue au dernier round avec $\sigma^2/\varepsilon_var = 0{,}95$ produit confidence $\approx 0{,}55$. Côté client, le seuil par défaut *DEFAULT_CONFIDENCE_FLOOR = 0,55* déclasse en severity *warn* tout CONVERGED inférieur.

8. efe_orch — orchestrateur de jobs

efe_orch pilote tous les *jobs* de fine-tuning. Il reçoit les soumissions, sélectionne un worker, supervise la progression via heartbeats, et consigne l'état en base. C'est la colonne vertébrale de la chaîne de build.

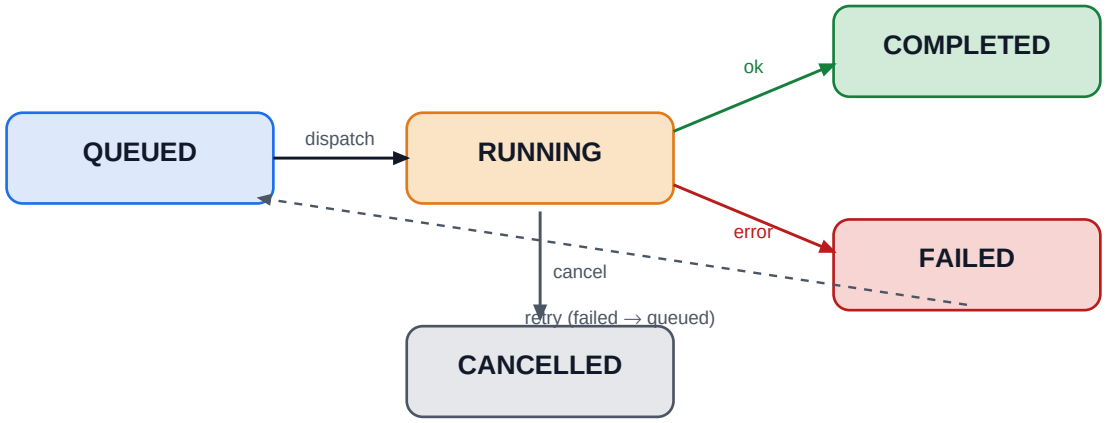


Figure 8 — FSM des jobs. Cinq états, transitions strictes. CAS optimiste sur state_version interdit les courses d'écriture. FAILED peut être retenté en QUEUED avec backoff exponentiel.

8.1 Sous-composants

Composant	Rôle
dispatcher	Sélection du worker (capacité, affinité tenant, taints)
scheduler	Priorisation file d'attente (tenant-fair, âge, priorité explicite)
state_machine	FSM stricte + CAS version (cf. figure 8)
heartbeat	Détection worker silencieux (timeout 60 s → requeue)
retry	Backoff exponentiel + jitter décorrélé (pattern AWS)
services	Règles métier haut-niveau (quotas, validations)
observability	Traces OTel + compteurs Prometheus + journaux d'audit

8.2 Backoff retry : tableau typique

Tentative	Délai nominal (ms)	Délai effectif avec jitter
1	1 000	750 – 1 250
2	2 000	1 500 – 2 500
3	4 000	3 000 – 5 000
4	8 000	6 000 – 10 000
5	16 000	12 000 – 20 000
≥ 6	60 000 (plafond)	45 000 – 75 000

9. efe_worker — fine-tuning (méthodes)

efe_worker exécute les jobs reçus. Il supporte cinq méthodes d'entraînement implémentées autour d'un protocole commun *Method*, ce qui rend l'ajout d'une sixième trivial.

9.1 Les cinq méthodes supportées

Méthode	Objectif	Hyperparamètres clés
lora	Adapter LoRA PEFT sur LLM gelé	r (1-64) · alpha · dropout · target_modules
classif	Classification multiclasse cross-entropy	lr · batch_size · weight_decay · epochs
infonce	Contrastive learning sur paires positives	τ (température) · negatives/batch
triplet	Triplet margin loss (anchor/pos/neg)	margin · miner · norm
distill	Distillation enseignant → élève (KL sur logits)	T (température) · α (balance) · top_k

9.2 Garde-fous anti-divergence en training

• **Loss non-finie** — détection par isfinite() → *TrainingDiverged*, job marqué FAILED sans retry.

• **Explosion du gradient** — norm clipping à 1,0 ; au-delà d'un seuil absolu (ex. 100) → TrainingDiverged.

• **Chute abrupte de la loss** → souvent un mode collapse en training ; alerte SRE.

• **Métrique de validation en stagnation** → early stopping.

9.3 Gestion d'artefacts produits

• Seul le résultat utile est sauvegardé : les **poids LoRA** (quelques Mo), pas le modèle de base.

• Le manifeste JSON décrit les hyperparamètres, la commit-SHA, la snapshot dataset et les métriques de validation.

• L'artefact est téléversé sur S3 (OCI-compatible) puis notifié à efe_registry qui le signe (cf. §10).

• Le worker ne signe pas lui-même — séparation des privilèges.

10. efe_registry — artefacts + signing (4 couches)

efe_registry catalogue les artefacts produits par efe_worker et orchestre la chaîne de signature + vérification. Trois méthodes de signature et quatre couches de vérification sont supportées.

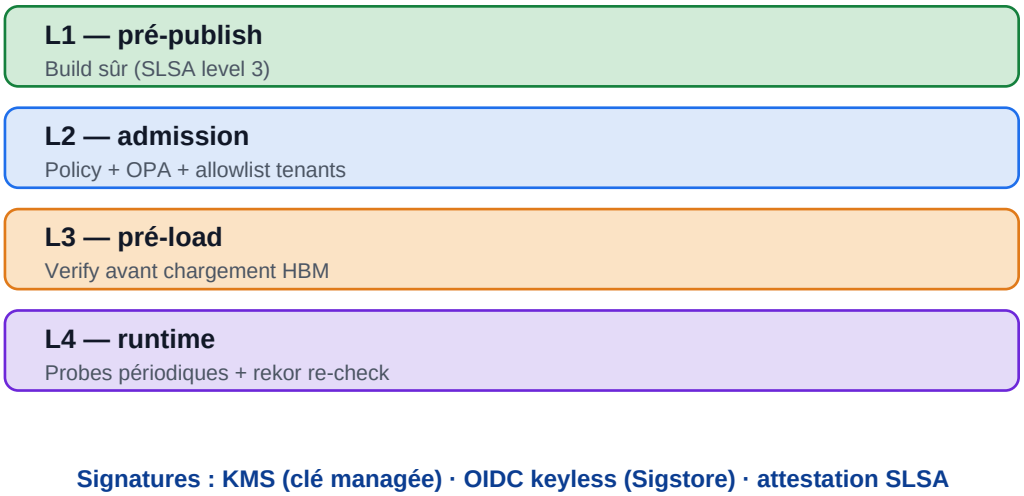


Figure 9 — Les 4 couches de vérification. L1 au build, L2 à l'admission dans le registry, L3 juste avant chargement en HBM, L4 en runtime continu. Une défaillance à n'importe quelle couche arrête la promotion / le chargement.

10.1 Les trois méthodes de signature

• **KMS (Key Management Service)** — clé managée par le cloud provider (GCP KMS, AWS KMS). Bonne intégration IAM mais dépendance fournisseur.

• **Keyless OIDC (Sigstore)** — certificat éphémère émis par Fulcio depuis un jeton OIDC (GitHub Actions, Workload Identity). Signature publiée dans Rekor (transparency log immutable).

• **Attestation SLSA** — attestation in-toto signée qui décrit comment l'artefact a été construit (builder, source, inputs).

10.2 Les quatre couches de vérification

Couche	Quand	Quoi
L1 — pré-publish	Avant le put dans le registry	Builder SLSA niveau 3, source tracée
L2 — admission	À l'entrée du registry	Policy OPA, tenant allowlist, format
L3 — pré-load	Juste avant chargement HBM	Signature + rekor_uuid + certificat valides
L4 — runtime	Probes périodiques	Re-vérification continue, détection de révocation

10.3 Modes de publication

• **stable** — artefact actif pour le trafic normal.

• **canary** — artefact visible uniquement à X % du trafic (cf. §12 efe_rollout).

• **shadow** — artefact évalué en miroir, jamais servi à l'utilisateur. Utile pour valider un nouveau modèle sur prod réelle sans risque.

• **deprecated** — conservé pour audit mais plus servi.

11. efe_gate — comparateur + décision

efe_gate décide si un **candidat** fraîchement entraîné peut remplacer le **champion** en production. La décision est statistiquement rigoureuse : chaque métrique est comparée par un test approprié, avec intervalles de confiance et p-values.

11.1 Les quatre tests statistiques supportés

Test	Usage	Statistique
welch_test	Moyennes de variances inégales	t de Welch approximé
mcnemar_test	Paires binaires succès/échec	χ^2 McNemar avec correction continuité
tost_equivalence	Test d'équivalence à un seuil δ	Two One-Sided Tests
bootstrap_delta_ci	CI sur différence non-paramétrique	Bootstrap percentile 10 000 tirages

11.2 Les six verdicts par métrique

- **PASS** — pas de régression, critère satisfait.
- **BETTER** — amélioration statistiquement significative.
- **FAIL** — régression significative sur une métrique critique.
- **AMBIGUOUS** — CI inclut zéro ; on ne peut pas trancher.
- **BUDGET_EXCEEDED** — dépassement d'un budget dur (latence, coût, énergie).
- **UNCOMPARABLE** — la métrique est absente d'un des deux bundles (souvent nouvelle métrique).

11.3 Logique de décision globale

La promotion globale n'intervient que si : (1) aucun BUDGET_EXCEEDED, (2) aucune métrique à rôle MUST_NOT_REGRESS en FAIL, (3) les métriques SECONDARY sont toutes PASS ou BETTER, (4) au moins une métrique PRIMARY est BETTER. Sinon la décision est HOLD (on reste sur le champion). Un REJECT explicite n'arrive qu'en cas de violation d'une règle obligatoire.

11.4 Rôles de métriques

- **PRIMARY** — métrique d'objectif (accuracy, F1) — au moins une doit s'améliorer.
- **SECONDARY** — métrique de support — ne doit pas régresser.
- **MUST_NOT_REGRESS** — métrique critique (ex. fairness) — zéro tolérance à la régression.
- **BUDGET** — coût, latence, énergie — plafond absolu.

12. efe_rollout — canari + bandits

efe_rollout déploie progressivement une nouvelle version en production. Deux stratégies sont implémentées : les **étapes canari** (discrètes : 1 % → 5 % → 25 % → 100 %) et les **bandits multi-armés** pour A/B tests à N variantes.

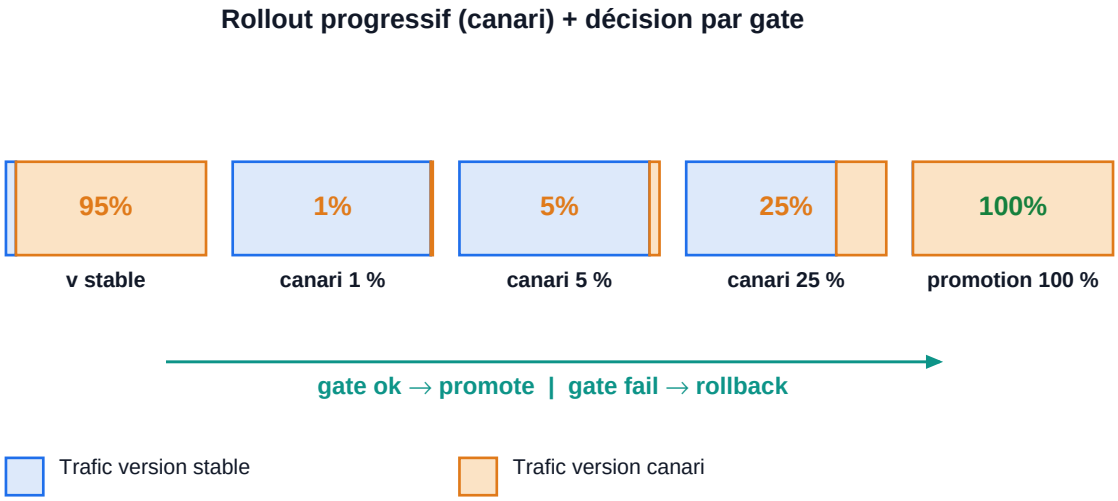


Figure 10 — Rollout progressif. À chaque étape, efe_gate ré-évalue le candidat sur le trafic canari ; si OK, on passe à l'étape suivante ; sinon, rollback automatique en < 30 s.

12.1 Les trois samplers de bandit

Sampler	Principe	Quand l'utiliser
Thompson	Échantillonner une récompense depuis Beta(α, β) par bras, choisir l'argmax	A/B binaire (conversion, success). Regret quasi-optimal.
UCB1	Score = moyenne + √(2 ln T / n) ; déterministe	Récompenses bornées continues. Simple, reproductible.
ε-greedy	P(ε) uniforme, sinon argmax moyenne ; ε décroît	Fallback simple. Moins bon regret.

12.2 Le traffic-split déterministe

Pour garantir qu'un même utilisateur reçoive toujours la même variante (cohérence perçue), le split utilise un hash déterministe : *user_hash = HMAC(DOMAIN_TAG_USER_ID, tenant || user_id) mod 10 000*. Si user_hash < split_ppm, variante canari, sinon stable. Même utilisateur, même session, même variante.

12.3 Layers d'expérimentation

Inspiré du Layered Experimentation Framework de Google, le sous-paquet *layers* permet d'empiler plusieurs expérimentations orthogonales sur le même trafic. Chaque layer consomme une portion des bits du user_hash. Cela garantit l'indépendance statistique entre expérimentations concurrentes.

12.4 Les quatre déclencheurs de rollback

- **gate fail** — métrique primaire régresse au-delà du seuil.
- **drift detection** — efe_drift signale un PSI > critical sur une feature clé.
- **error rate** — taux d'erreurs 5xx × 10 vs baseline.
- **budget breach** — latence p95 ou coût sortent du SLO.

13. efe_inference — serving et backends

efe_inference est la surface d'exécution en production. Il est agnostique au modèle : un registre de **backends pluggables** le connecte indifféremment à des services externes (Claude, OpenAI, Mistral), à un serveur interne (vLLM), ou à un adaptateur PEFT local.

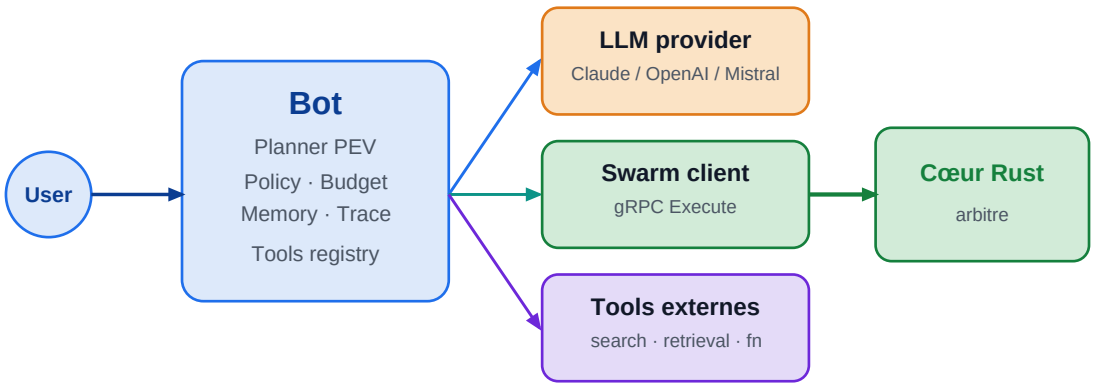


Figure 11 — Interactions bot + LLM + swarm. Le bot orchestre trois canaux : un provider LLM pour la planification, le Swarm client pour l'exécution en cœur Rust, et le registre de tools externes pour les actions (search, retrieval, function calling).

13.1 Les six backends implémentés

Backend	Type	Usage
claude	frontier	Anthropic Claude via SDK
openai	frontier	OpenAI GPT via SDK officiel
mistral	frontier	Mistral hosted ou local
vllm	self_hosted	Serveur vLLM interne (haute concurrence)
peft	self_hosted	Adapter LoRA chargé via efe_registry
mock	test	Déterministe, offline, pour tests

13.2 Mécanismes de résilience

- **Circuit breaker** par backend (open/half-open/closed) avec fenêtre glissante sur taux d'erreur.
- **Timeout par requête** propagé au niveau SDK.
- **Retry avec backoff** — maximum 2 tentatives, pas plus.
- **Fallback désigné** — si le backend primaire échoue, basculement vers un backend secondaire avec dégradation annoncée.

13.3 Post-flight de sécurité

- **Filtrage PII** — regex + modèle léger, remplacement par tokens placeholders.
- **Limite de longueur** — troncature gracieuse + avertissement.

• **Détection de patterns indésirables** — injections, contenus interdits par politique tenant.

• **Comptabilisation** — tokens + coût par tenant → efe_inference/accounting.

14. efe_drift — pipeline de détection

efe_drift consomme le flux d'événements d'inférence et détecte en continu toute dérive entre la distribution actuelle et une **baseline scellée**. Sept familles de détecteurs sont implémentées en **Python pur** (pas de SciPy) pour la portabilité et la reproductibilité bit-pour-bit.

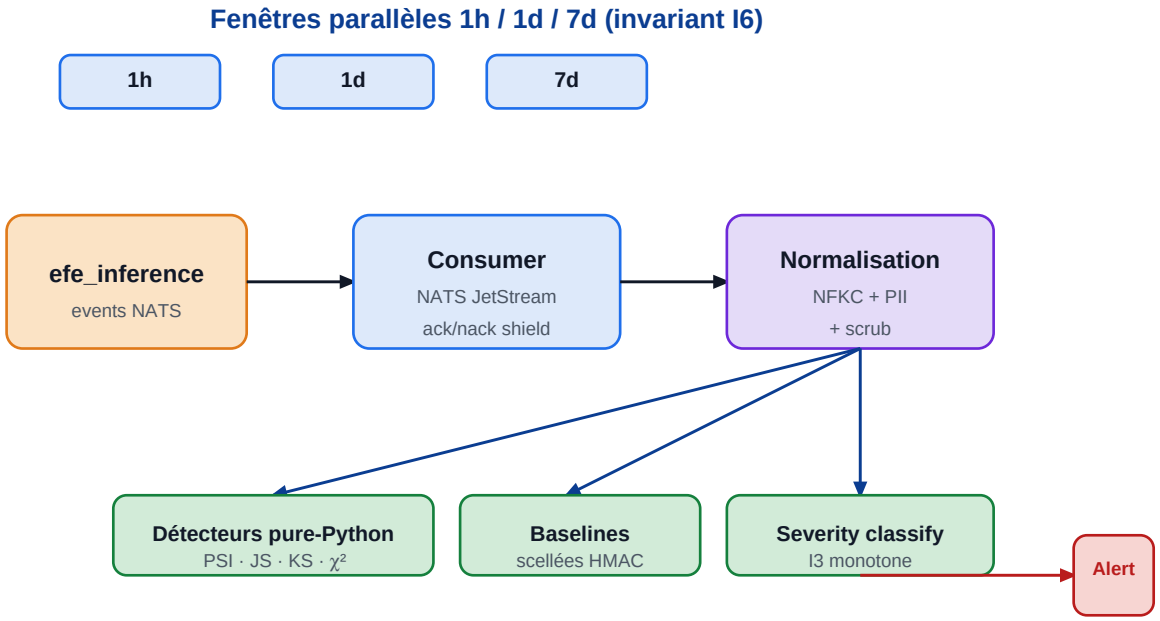


Figure 12 — Pipeline de détection. Les events sont consommés depuis NATS, normalisés (NFKC + PII), puis évalués en parallèle sur trois fenêtres temporelles (1h, 1d, 7d) contre les baselines scellées. La sévérité est classifiée (monotone I3), et une alerte est routée vers les canaux configurés.

14.1 Les sept familles de détecteurs

Famille	Mesure	Orientation
PSI	Population Stability Index (Σ (p-q)·ln(p/q))	plus grand = pire
JS	Jensen-Shannon divergence, symétrique [0, ln 2]	plus grand = pire
KS	Kolmogorov-Smirnov à deux échantillons	plus grand = pire
χ²	χ² de contingence, sévérité sur p-value	plus petit = pire (p-value)
Wasserstein	Wasserstein-1 (Earth Mover's Distance)	plus grand = pire
MMD	Maximum Mean Discrepancy avec noyau RBF	plus grand = pire
classifier	AUC d'un classifieur baseline-vs-current (Rabanser 2019)	plus grand = pire

14.2 Classification de sévérité monotone (I3)

- Quatre niveaux : **none, warning, major, critical**.
- La fonction score → sévérité est strictement monotone dans l'orientation choisie (invariant I3).
- Les seuils par défaut sont issus de la littérature : PSI 0,10 warning / 0,25 major / 0,40 critical.
- Validation des seuils à la construction : une inversion accidentelle (warning > critical) lève ValueError.

14.3 Multi-échelle temporel (invariant I6)

Chaque évaluation est calculée en **parallèle** sur trois fenêtres : 1h (réactivité), 1d (stabilité journalière) et 7d (tendance hebdomadaire). Une dérive qui n'apparaît qu'en 1h est souvent un incident ponctuel ; celle qui persiste en 7d est structurelle. La corrélation de ces trois signaux guide la priorité d'intervention.

15. efe_platform — bot et SDK

efe_platform est le SDK public + le **bot** orchestrant des requêtes complètes. L'API exposée est volontairement minimale : *bot.answer(prompt, tenant=...)* suffit pour obtenir une réponse avec trace et verdict. Derrière, neuf sous-systèmes injectés exécutent le PEV (Plan / Execute / Verify).

15.1 Les neuf sous-systèmes du bot

Sous-système	Rôle
SwarmClient	Exécute la RPC Execute() auprès du cœur Rust
ConeRegistryClient	Résout les ConeRef en AlignmentCone (cache TTL 30 s)
LlmProvider	Backend de planification (claude | openai | mistral | mock)
ToolRegistry	Catalogue des tools (search, retrieval, function)
ConversationMemory	Contexte borné, rolling window
BudgetLedger	Garde-fou coût (tokens, $/mois, compute)
PolicyEngine	Préflight safety (prompt injection, tenants blocklist)
Planner	FSM PEV : décomposer prompt → plan_step[]
AuditWriter	Journal scellé HMAC (DOMAIN_TAG_AUDIT)

15.2 Cycle PEV (Plan / Execute / Verify)

• **Plan** — l'LlmProvider propose une séquence d'étapes (embed, tool call, swarm execute).

• **Execute** — chaque étape est dispatchée : le swarm pour l'inférence, les tools pour les actions.

• **Verify** — le verdict du swarm + les post-flight safety déterminent l'acceptation de la réponse.

• Le tout est consigné dans la **trace** (liste d'événements typés : plan_step, tool_call, swarm_execute, policy_check, budget_hold, audit_seal).

15.3 Hiérarchie d'erreurs

• Racine unique : *EfePlatformError*.

• Attribut *retryable: bool* consommé par la boucle de retry.

• Code *snake_case* stable pour logs et audit (ex. budget_exceeded, swarm_unavailable, tool_failed).

• Sérialisation canonique via *to_dict()* pour réponses HTTP 5xx (Problem Details RFC 7807).

16. Sécurité : zones de confiance

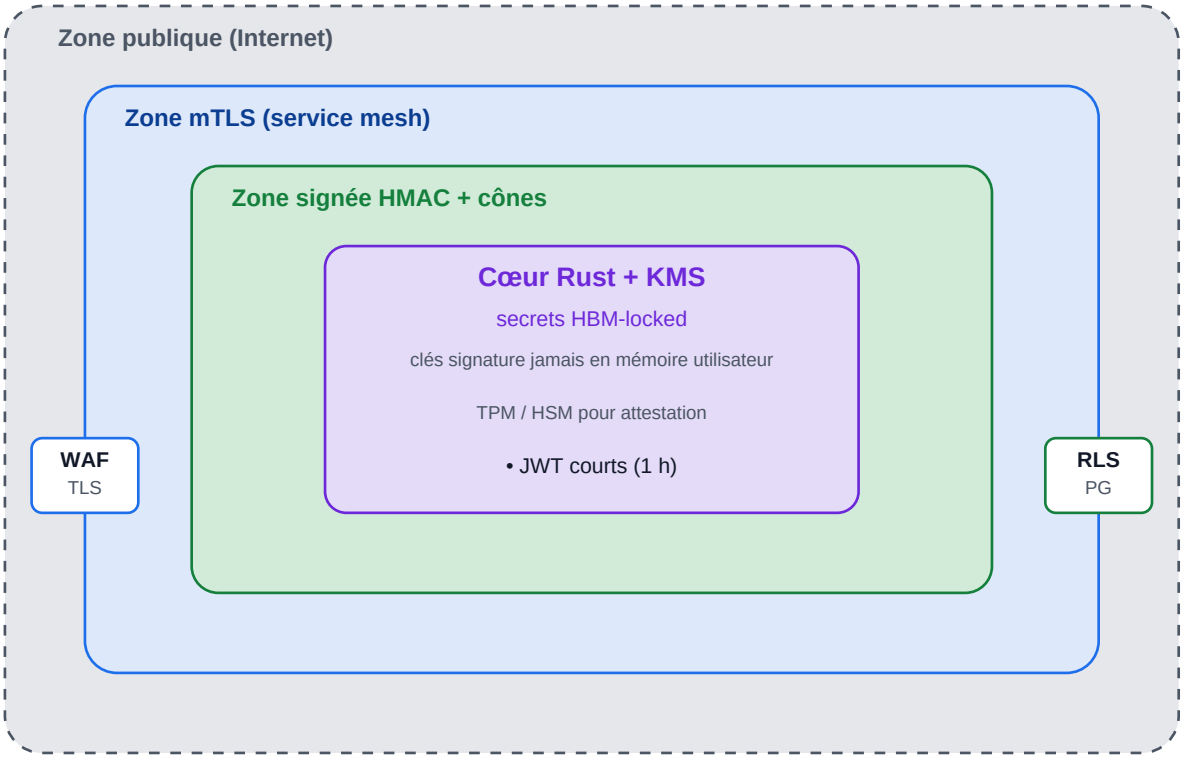


Figure 13 — Zones de confiance concentriques. De l'extérieur : Internet (gris), service mesh mTLS (bleu), zone signée HMAC + cônes (vert), cœur Rust + KMS (violet). Chaque franchissement exige une authentification et ajoute des contrôles.

16.1 Huit domaines de sceaux HMAC (I8)

Domain tag	Usage
efe-drift/baseline/v1	Scellement d'une baseline de feature
efe-drift/nats/v1	Signature des messages NATS sortants
efe-drift/webhook/v1	Signature payload webhook X-EFE-Signature
efe-drift/user-id/v1	Hash stable user-id pour traffic split
efe-registry/artifact/v1	Sceau du manifest d'artefact
efe-gate/decision/v1	Sceau de décision de gate
efe-orch/job/v1	Sceau de commande worker
efe-platform/catalog/v1	Sceau du catalogue versionné

16.2 Pourquoi les domain tags

Un HMAC sans domain tag est vulnérable à une attaque **cross-protocol** : un payload signé pour un canal peut être rejoué comme valide sur un autre canal si la même clé est partagée. Le *domain separation tag* préfixe le message HMAC d'une constante unique par usage, garantissant qu'aucun sceau émis pour « NATS » ne peut jamais être accepté pour « webhook », même avec la même clé.

16.3 Autres contrôles actifs

- **mTLS** entre tous les services (Istio / Linkerd).

• **JWT courts (1 h)** + refresh tokens dédiés, claims tenant_id + scopes.

• **RBAC** à trois rôles : admin, operator, reader.

• **Row-Level Security PostgreSQL** — policy tenant_id = current_setting('app.tenant_id') sur toutes les tables multi-tenant.

• **Quotas token-bucket** par tenant pour rps et coût.

• **Scan gitleaks** en CI contre les secrets committés.

• **External Secrets Operator** pour synchroniser les secrets depuis AWS Secrets Manager / Vault.

17. Observabilité : les trois piliers

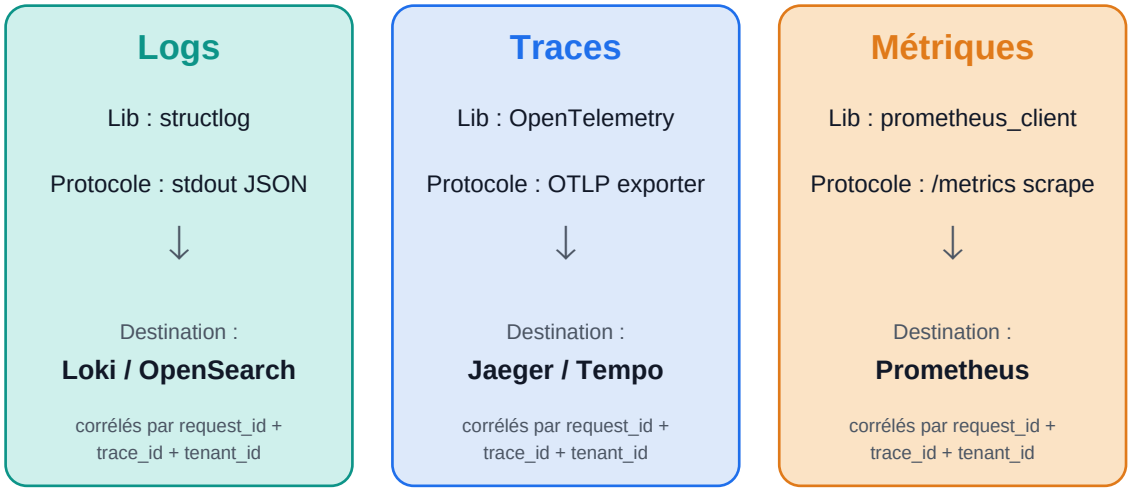


Figure 14 — Les trois piliers d'observabilité. Logs (structlog → Loki), Traces (OpenTelemetry → Jaeger), Métriques (Prometheus client → Prometheus). Tous corrélés par request_id + trace_id + tenant_id.

17.1 Logs structurés

- Format JSON strict, une ligne par événement.
- Clés obligatoires : timestamp (RFC 3339), level, service, request_id, trace_id, tenant_id, task_id.
- Propagation par structlog.contextvars : une fois bound sur une coroutine asyncio, tous les logs émis depuis cette tâche portent automatiquement les mêmes clés.
- Destination : stdout → Fluent Bit → Loki (ou OpenSearch selon le cluster).

17.2 Traces distribuées

- Standard OpenTelemetry (OTLP).
- Auto-instrumentation : gRPC, HTTP, SQLAlchemy, httpx.
- Spans manuels sur les méthodes critiques : arbiter.round, hmac.seal, gate.compare, drift.evaluate.
- Exporter OTLP → OpenTelemetry Collector → Jaeger (ou Tempo).
- Sampling : head-based 100 % des erreurs + 10 % du trafic normal ; tail-based par erreur inhabituelle.

17.3 Métriques

- Format Prometheus exposé sur /metrics par chaque pod.
- Convention de nommage : *efe_<service>_<measure>_<unit>*.
- Counters : total d'événements discrets.
- Histograms : durées et distributions (buckets adaptés).
- Labels minimaux — jamais user_id ou request_id (cardinalité explosive).

• Dashboards Grafana as-code versionnés dans le dépôt.

17.4 Quelques SLO MVP

Service	SLI	SLO MVP
efe_inference	Disponibilité 2xx	99,5 %
efe_inference	Latence p95	< 600 ms
efe_drift	Détection → alerte	< 60 s
efe_orch	Démarrage job p95	< 15 s
efe_registry	Verify L3 p99	< 200 ms
efe_gate	Décision p95	< 2 s

18. Cycle de vie complet (flux e2e)

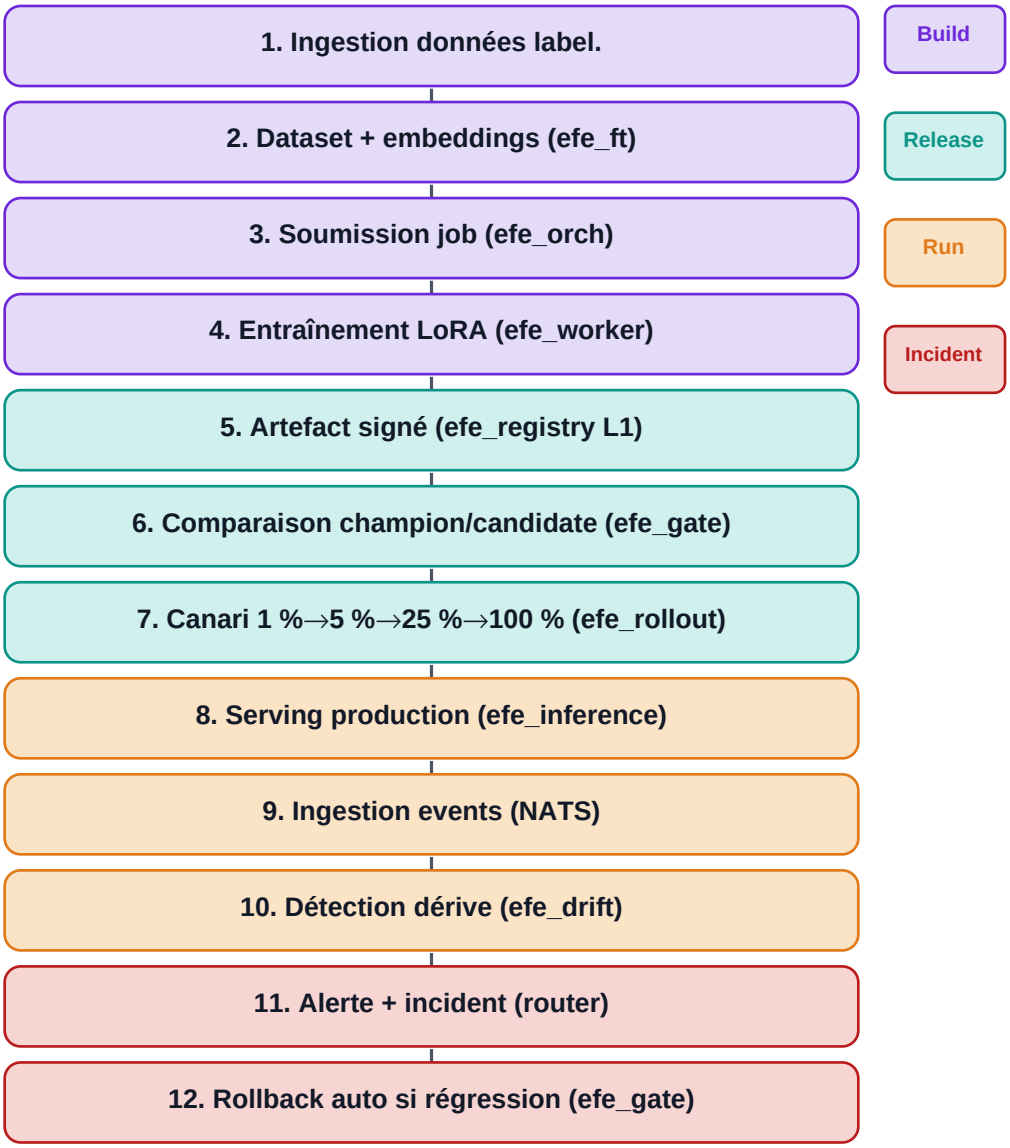


Figure 15 — Flux complet bout-à-bout en 12 étapes. Violet = chaîne de build, bleu-vert = chaîne de release, orange = chaîne de run, rouge = incident. Les étapes 1-4 sont asynchrones ; 8-10 sont continues.

18.1 Les 12 étapes en détail

• **1. Ingestion de données labellisées** — un labeler Human-In-The-Loop ou automatique pousse un dataset versionné vers efe_ft.

• **2. Construction du dataset + embeddings** — efe_ft encode, déduplique, stratifie, shuffle déterministe.

• **3. Soumission du job** — efe_orch reçoit la spécification (méthode, hyperparamètres, budget).

• **4. Entraînement** — efe_worker sélectionne la méthode (LoRA, distill, etc.), entraîne, émet des checkpoints.

• **5. Artefact signé** — efe_registry reçoit les poids, les signe (L1), les stocke OCI, publie rekor_uuid.

• **6. Comparaison champion/candidate** — efe_gate compare métrique par métrique sur un split de validation.

• **7. Rollout progressif** — efe_rollout démarre à 1 % du trafic, observe, escalade si gate ok.

• **8. Serving production** — efe_inference sert les requêtes utilisateurs avec resilience + post-flight safety.

• **9. Ingestion events** — chaque réponse émet un event NATS sur efe.inference.events.

• **10. Détection dérive** — efe_drift consomme et applique ses 7 détecteurs × 3 fenêtres temporelles.

• **11. Alerte + incident** — dépassement critical → alerte routée (NATS, webhook, log) + incident ouvert.

• **12. Rollback auto** — si régression confirmée, efe_gate déclenche un retour au champion en < 30 s.

18.2 Garanties transverses du flux

• **Idempotence** — même request_id → même réponse (cache + dédup event_id).

• **Traçabilité** — toute étape porte trace_id OTel + request_id.

• **Intégrité** — artefacts signés à chaque transition.

• **Isolation** — tenant_id bout-à-bout, RLS + cônes par tenant.

• **Budgets** — tout dépassement wall-clock / énergie / HBM termine proprement.

19. Déploiement Kubernetes

EFE est déployé sur un cluster Kubernetes managed. Les services sont répartis en quatre namespaces qui correspondent aux quatre profils de charge (contrôle, runtime chaud, batch GPU, cœur Rust GPU dédié).

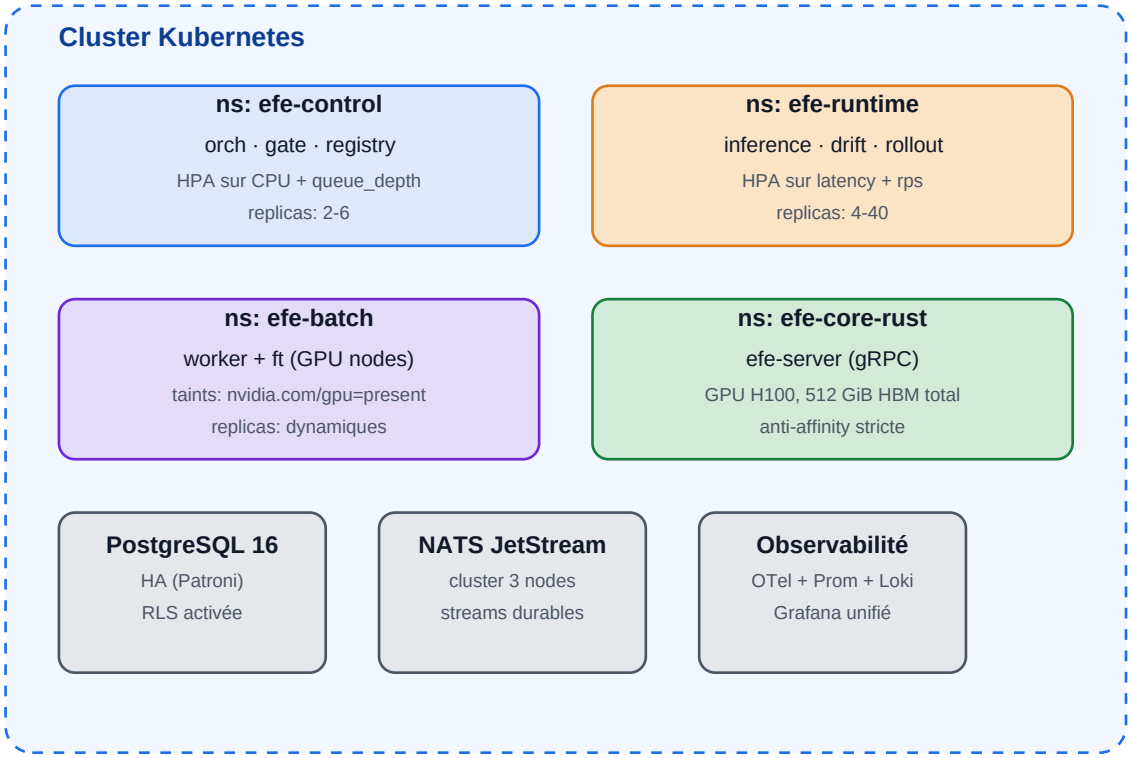


Figure 16 — Topologie Kubernetes. Quatre namespaces : efe-control (plan de contrôle), efe-runtime (plan de données chaud), efe-batch (GPU nodes pour training), efe-core-rust (GPU H100 pour l'arbitre Rust). Stateful services partagés en bas.

19.1 Règles de scheduling

- **Anti-affinity stricte** sur le cœur Rust : jamais deux pods sur le même nœud (résilience GPU).
- **Taints/tolerations** : les nodes GPU sont tainted *nvidia.com/gpu=present:NoSchedule*, seuls les pods tolérants y sont placés.
- **Topology spread constraints** sur 3 zones : pas plus de 1/3 des replicas dans une même AZ.
- **Node affinity** par classe de nœud (cpu-heavy pour les services contrôle, gpu-heavy pour le cœur).

19.2 Autoscaling (HPA)

Service	Métrique de scale	Seuil
efe_inference	requests_queue_depth custom	scale out à > 50
efe_inference	CPU util	> 70 %
efe_orch	pending_jobs custom	> 20
efe_worker	queued_gpu_jobs	> 2
efe_drift	events_lag NATS	> 5 s
efe_gate	pending_comparisons	> 10

19.3 Release pipeline (CI/CD)

- PR → GitHub Actions → just fmt + just lint + just test.
- Merge main → build multi-arch images (amd64 + arm64) → signe avec cosign keyless.
- Helm chart upgrade via ArgoCD (GitOps).
- Progressive rollout via Argo Rollouts + Flagger (intégré à efe_rollout).
- Chaque release publie un SBOM (CycloneDX) attaché à l'image.

20. Critères d'acceptation MVP

20.1 Qualité logicielle (gates CI)

- **ruff format --check** — 100 % propre.
- **ruff check** — règles E/F/W/I/B/UP/SIM/N/ANN/RUF.
- **cargo clippy -- -D warnings** — zéro warning Rust.
- **mypy --strict** — zéro erreur, zéro type: ignore sans motif.
- **pytest** — 100 % verts sur chaque service.
- **Couverture** — ≥ 90 % global, ≥ 95 % efe_platform/swarm.
- **just test-smoke** — < 10 s, couvre les 9 services.

20.2 Performance

Capacité	Cible MVP
Latence swarm.execute p95	< 600 ms
Latence swarm.execute p99	< 1 200 ms
Ingestion d'événements / pod	≥ 5 000 evt/s
Débit inference (peft) / pod	≥ 200 rps sur T4
Détection dérive sur 10 k events	< 30 s
Rollback rollout	< 30 s après signal
Verify artefact L3	< 200 ms p99
Démarrage job p95	< 15 s
Consensus essaim (rounds)	typiquement 3-6

20.3 Sécurité

- Tous les HMAC ont un domain separation tag (I8).
- Artefacts signés L1 + vérifiés L3 obligatoire.
- PII linter actif (NFKC + depth-limit).
- JWT ≤ 1 h + refresh token séparé.
- RLS PostgreSQL sur toutes les tables multi-tenant.
- Scan gitleaks en CI.
- Revue 2-of-N sur toute PR touchant efe-core ou efe-ffi.
- Alertes sur toute régression de couverture des gates signing.

20.4 Documentation

- Chaque paquet a docstring + __all__ explicite.
- Chaque service a un runbook.md avec 10 incidents probables.
- ADR pour les 12 choix majeurs (Rust core, gRPC, pure-Python stats, signing keyless, cônes, etc.).
- SLO déclarés dans docs/slo.md.

- Documentation publique mkdocs sur docs.efe.ai à chaque release.
- Schémas d'architecture mis à jour à chaque ADR.

Fin du document. *Ce dossier constitue la référence technique illustrée du projet EFE pour le MVP. Il doit être relu à chaque rupture d'invariant, ajout de template d'agent, ou changement majeur de contrat gRPC. Toute question technique doit être remontée au mainteneur désigné du service concerné.*